

Scikit-ReducedModel

Modelos de orden reducido con partición de dominio automatizada

Franco Cerino
Agustín Rodríguez-Medrano

Organización de la presentación

1. Introducción
2. Marco teórico
3. Principios de Scikit-Learn
4. Diseño del paquete
5. Ejemplo de aplicación
6. Conclusiones



Introducción

- Problemática principal: alto costo computacional de simulaciones numéricas para realizar estudios.
- Área de relatividad general: obtener expresiones de ondas gravitacionales de sistemas binarios compactos puede costar meses utilizando supercomputadoras.
- Estudios de estimación de parámetros pueden requerir hasta millones de estimaciones secuenciales, dominando el costo computacional del problema.
- Necesidad de metodologías que mejoren radicalmente el tiempo de evaluación.

Introducción

- Construcción de **modelos sustitutos** de ecuaciones diferenciales, aprovechando la redundancia de las soluciones con respecto al espacio de parámetros.
- Extraen la información más relevante de simulaciones de alto costo computacional y la utilizan para aprender los patrones de las soluciones.

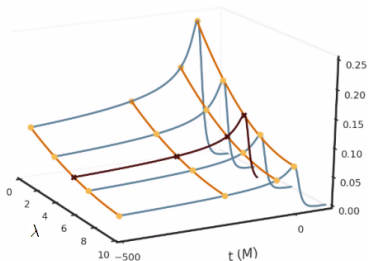


Figura 1: Esquema de un modelo sustituto.

Introducción

- Creamos un paquete que implementa todas las etapas de los modelos sustitutos, facilitando estudios en diversas áreas de ciencia e ingeniería.
- Se desarrolló con un enfoque en *Scikit-Learn*, que ayuda a construir modelos personalizados fácilmente.



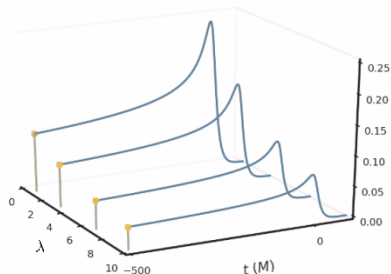
La construcción de un modelo sustituto conlleva tres pasos secuenciales:

- 1) Encontrar una base reducida del espacio de soluciones (preprocesamiento).
- 2) Construir un interpolante empírico (preprocesamiento).
- 3) Realizar regresiones para poder predecir nuevas soluciones (aprendizaje).

Marco teórico: 1) Bases reducidas

- Espacio de funciones $\mathcal{F} := h_\lambda(t)$, parámetro λ en un dominio $\mathcal{D}$.
- Finalidad: construir una base representativa de $\mathcal{F}$ con la menor cantidad de información posible.
- Funciones de entrenamiento fidedignas $\mathcal{T} := \{h_{\lambda_i}\}_{i=1}^m$.
- Algoritmo greedy: en cada paso el elemento del conjunto de entrenamiento peor representado por la base se agrega a la misma.
- Finalización del algoritmo: n_{max} o representar $\mathcal{T}$ con tolerancia ϵ .

$$h_\lambda(t) \approx \sum_{i=1}^n c_i(\lambda) e_i(t),$$



Marco teórico: 1) Partición de dominio con hp-greedy

- Mejorar tiempo de predicción particionando el espacio de parámetros.
- Se construye una base reducida y si no representa a $\mathcal{T}$ a una tolerancia ϵ , se procede a particionar recursivamente.
- Partición adaptativa a los datos de entrenamiento.

Criterios para terminar de particionar:

- Llegar a tolerancia ϵ deseada.
- Alcanzar cantidad máxima de particiones locales l_{max} .

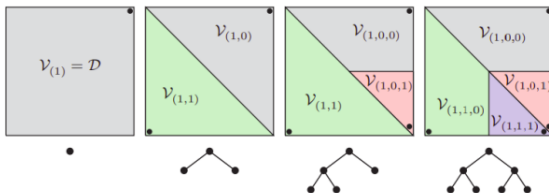


Figura 2: Particiones y estructura de arbol obtenida con hp-greedy.

Marco teórico: 2) Método de interpolante empírico

- A partir de una base reducida encontrada, se emplea un algoritmo greedy para seleccionar los tiempos más relevantes, denominados *tiempos empíricos*: $\{T_i\}_{i=1}^n$
- En el mismo procedimiento se obtiene una expresión para un *interpolante empírico* que interpola en $\{T_i\}_{i=1}^n$.

$$\mathcal{I}[h](\lambda; t) := \sum_{i=1}^n B_i(t) h(\lambda, T_i) ,$$

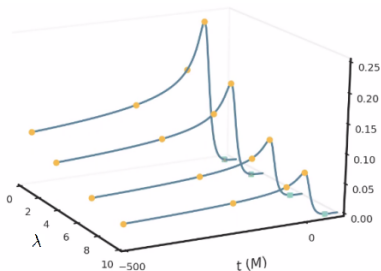


Figura 3: Los puntos amarillos representan las funciones en los T_i encontrados.

Marco teórico: 3) Regresiones para el modelo sustituto

- Para obtener una expresión de una solución de un interpolante empírico para un λ arbitrario, es necesario tener un valor de $h(\lambda, T_i)$ para todo T_i .
- Para cada T_i se toma los valores $\{h(\lambda_j, T_i)\}_j$ del conjunto de entrenamiento para entrenar un algoritmo de machine learning y poder tener una expresión $h_i^{\text{ML}}(\lambda, T_i)$ para todo λ en $\mathcal{D}$.

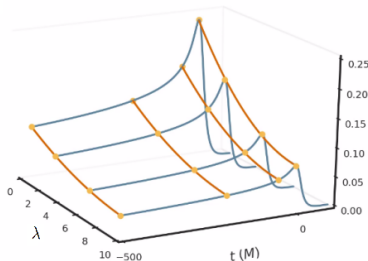


Figura 4: Curvas amarillas: regresiones en los tiempos empíricos T_i .

Marco teórico: Computando una predicción

$$h_{\text{ML}}(\lambda; t) := \sum_{i=1}^n B_i(t) h_i^{\text{ML}}(\lambda, T_i),$$

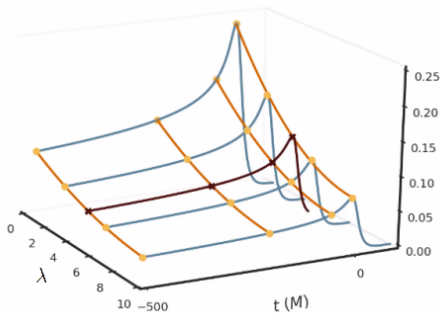


Figura 5: La curva negra representa una predicción obtenida al evaluar las regresiones en el λ dado.

Principios de Scikit-Learn

- Nos basamos en la idea de diseño de la librería **Scikit-Learn**.
- Interfaz fácil de usar, eficiente y bien documentada para algoritmos de aprendizaje automático en Python.

Principios de diseño:

- **No proliferación de clases.** Los algoritmos de aprendizaje son los únicos objetos que se deben representar mediante una clase de Python.
- **Composición.** Si la constitución de un algoritmo de *machine learning* es posible interpretar a partir de un conjunto de componentes fundamentales, representarlo de esta forma.
- **Valores por default.** Existencia parámetros definidos por default para cada algoritmo.

Al igual que `Scikit-Learn`, el uso de `Scikit-ReducedModel` se basa en aplicar tres interfaces complementarias:

- **estimadores**: definen la instanciación de objetos con sus respectivos hiperparámetros y exponen un método `fit`.
- **transformadores**: toman como datos como input y obtienen una versión transformada de ellos, a través del método `transform`.
- **predictores**: extienden la noción de estimadores, agregando un método `predict`.

Diseño de Scikit-ReducedModel

- Se ha desarrollado un paquete *open-source* de modelos de orden reducido, utilizable en diversas áreas científicas.
- La API pública de **Scikit-ReducedModel** consta de tres módulos principales: `reducedbasis`, `empiricalinterpolation` y `surrogate`. Cada uno tiene una clase homónima que implementa cada metodología.
- Las clases son estimadores, utilizan el método `fit` para entrenarse.
- Utilizan el método `transform` o `predict`, según corresponda, para obtener representaciones de funciones.

Diseño de Scikit-ReducedModel

- Cada módulo implementa una etapa para construir un modelo sustituto, el cual se obtiene a través de la composición de ellos.
- La modularización del proyecto permite realizar estudios específicos de bases reducidas o el método de interpolación empírico fácilmente.

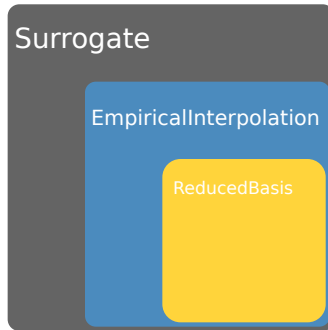


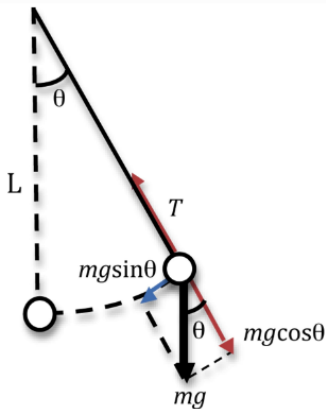
Figura 6: Esquema representativo de las clases de `Scikit-ReducedModel`. 14/23

Función `mksurrogate`:

- Es un patrón factory: utiliza una serie de clases y toma toda la responsabilidad para construir una instancia de una nueva clase.
- Se le debe otorgar datos de entrenamiento y se puede especificar hiperparámetros de cualquiera de las clases involucradas en conformar un modelo sustituto.
- Devuelve un modelo sustituto listo para realizar predicciones, permitiendo una interfaz simple al usuario y evitando tener que inicializar todas las clases por separado.

Ejemplo de aplicación

En este breve ejemplo, se construye una base reducida, un interpolante empírico y un modelo sustituto para un péndulo simple, obteniendo las respectivas aproximaciones



Primero se construye una expresion teórica para un péndulo simple amortiguado.

```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> from scipy.integrate import odeint
>>> def pend(y, t, b, λ):
>>>     θ, σ = y
>>>     dydt = [σ, -b*σ -
>>>             λ*np.sin(θ)]
>>>     return dydt
```

```
>>> b = 0.2 # fricción
>>> y0 = [np.pi/2, 0.] # condición inicial
>>> times = np.linspace(0,50,1001)
```

A partir de expresiones teóricas se construye un conjunto de entrenamiento (training_set)
para un conjunto de parámetros (λ_train).

```
>>> λ_train = np.linspace(1,5,101)
>>> training_set = []
>>> for λ in λ_train:
>>>     sol = odeint(pend,y0, times, (b,λ))
>>>     training_set.append(sol[:,0])
>>> training_set = np.array(training_set)
```

Solución de test.

```
>>> λ_test = 2.94
>>> sol_test = odeint(pend, y0, times, (b,λ_test))
```

Se importa la clase ReducedBasis y se la instancia con hiperparámetros para contruir una base reducida.

```
>>> from skreducedmodel.reducedbasis import ReducedBasis
>>> rb = ReducedBasis(greedy_tol=1e-16,
>>>                    lmax=2,
>>>                    nmax=10
>>>                    )
>>> rb.fit(training_set=training_set,
>>>         parameters= $\lambda$ _train,
>>>         physical_points=times
>>>         )
>>> sol_rb = rb.transform(sol_test,  $\lambda$ _test)
```

Se accede a la base reducida en diferentes hojas del tree mediante el método rb.tree.leaves
ej: coloreamos los parametros de entrenamiento de cada partición.

```
>>> np.random.seed(seed=4)
>>> for leaf in rb.tree.leaves:
>>>     color = np.random.rand(3,)
>>>     for p in leaf.train_parameters[:,0]:
>>>         plt.axvline(p, c=color, lw=.5)
```

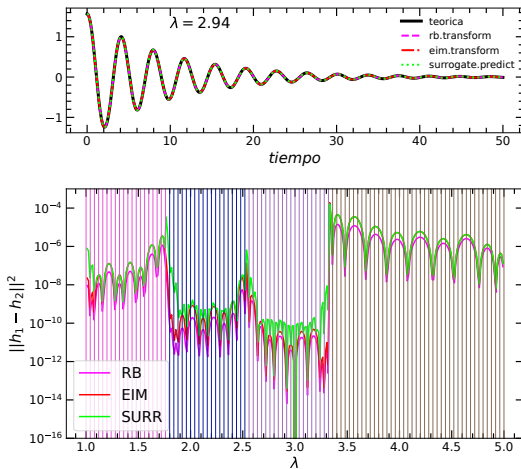
Se construye el interpolante empírico.

```
>>> from skreducedmodel.empiricalinterpolation import EmpiricalInterpolation
>>> eim = EmpiricalInterpolation(rb)
>>> eim.fit()
>>> sol_eim = eim.transform(sol_test,  $\lambda_{\text{test}}$ )
```

Se construye el modelo sustituto.

```
>>> from skreducedmodel.surrogate import Surrogate
>>> surrogate = Surrogate(eim,
>>>                       poly_deg=3
>>>                       )
>>> surrogate.fit()
>>> sol_surr = surrogate.predict( $\lambda_{\text{test}}$ )
```

Ejemplo de aplicación



Si el usuario solamente va a realizar predicciones con modelos sustitutos, puede proceder
de forma más sencilla utilizando la función factory mkssurrogate:

```
>>> from skreducedmodel.mksurrogate import mkssurrogate
>>> surrogate = mkssurrogate(
>>>     training_set=training_set,
>>>     parameters= $\lambda_{\text{train}}$ ,
>>>     physical_points=times
>>>     greedy_tol=1e-16,
>>>     lmax=2,
>>>     nmax=10
>>> )
>>> sol_surr = surrogate.predict( $\lambda_{\text{test}}$ )
```

Conclusiones

- Se lanza la versión 1,0 de **Scikit-ReducedModel**, el cual es un software para construir aproximaciones de espacios de funciones muy precisas y de rápida evaluación.
- El paquete brinda la posibilidad de hacer estudios separados con cada uno de los pasos de preprocesamiento, construyendo modelos personalizados y con la posibilidad de usarlos para crear nuevos modelos sustitutos.
- Se implementa el refinamiento *hp-greedy*, el cual es una técnica para poder construir aproximaciones de menor dimensionalidad y dar lugar a menor tiempo de evaluación.
- **Scikit-ReducedModel** fue diseñado en base a los estándares de **Scikit-Learn**, haciendo incapié en usabilidad sencilla.
- Estándares de estilo y documentación de código. Read The Docs.
- Coverage de 96 %, utilizando tox e integracion continua.
- Licencia MIT.
- Instalación: `pip install Scikit-ReducedModel`

Conclusiones

- Como trabajo futuro queda pendiente agregar mayor flexibilidad al paquete al momento de realizar las regresiones para un modelo sustituto. Este paso es el que mayor error introduce el modelo.
- Otro desarrollo a futuro es permitir modelos sustitutos con un espacio de parámetros multidimensional, que tendría una aplicación directa por ejemplo en modelos de coalescencia de sistemas binarios de agujeros negros con spin arbitrario.
- El paquete se puede enviar para ser introducido como parte de la librería **Scikit-Learn**.

This is it

